

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JEEVA	Reg. No.:	192421334
Section / Batch:	B.TECH IT	Date:	24.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q13. Smart Warehouse Storage Optimization (Bin Packing)

A warehouse must efficiently allocate packages of varying sizes into storage units with fixed capacity. Analyze how this problem can be modeled using Bin Packing algorithms. Identify constraints such as space utilization and item distribution. Design a feasible solution using greedy heuristics like First-Fit or Best-Fit. Discuss trade-offs between optimality and computational efficiency. Evaluate algorithm performance in large-scale warehouses. Analyze time complexity and justify solution effectiveness in logistics applications.

Code of Conduct JKM

I JEEVA 192421334 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Smart Warehouse Storage Optimization Using Bin Packing Algorithms

Abstract:

Smart warehouse storage optimization aims to allocate packages of different sizes into storage units with fixed capacities while minimizing unused space and the number of storage units required. This problem can be modeled as the **Bin Packing Problem**, where packages are treated as items and storage units are treated as bins. This report analyzes the major constraints of warehouse storage, including storage capacity, space utilization, item distribution, package compatibility, and accessibility. Greedy algorithms such as **First-Fit (FF)** and **Best-Fit (BF)** are examined as practical solutions. The **First-Fit Decreasing (FFD)** method is also considered, where packages are first sorted in decreasing order of size before applying First-Fit. These approaches provide good solutions with significantly lower computational requirements than exact optimization methods. The report analyzes their time complexity, advantages, limitations, and performance in large-scale warehouse environments. Although greedy algorithms do not always guarantee an optimal solution, their simplicity, speed, scalability, and good space utilization make them highly suitable for practical logistics and warehouse management applications.

Keywords: Bin Packing, Smart Warehouse, Storage Optimization, First-Fit, Best-Fit, First-Fit Decreasing, Space Utilization, Logistics

1. Introduction:

Efficient storage allocation is an important requirement in modern warehouse and logistics management. A warehouse receives packages of different sizes and must assign them to storage units with fixed capacities. The main objective is to use the available storage space efficiently while minimizing the number of storage units required.

This problem can be modeled as the **Bin Packing Problem**, a well-known combinatorial optimization problem. In this model, each package is considered an item with a specific size, and each storage unit is considered a bin with a fixed capacity.

Bin Packing algorithms are useful in warehouses because they can help determine how packages should be distributed among available storage units. Greedy algorithms such as **First-Fit**, **Best-Fit**, and **First-Fit Decreasing** can produce good solutions quickly without the high computational cost of exact optimization methods.

2. Problem Definition:

Consider a warehouse containing packages.

Let:

= number of packages

= size of package

= capacity of each storage unit

= number of storage units used

Each package must be assigned to exactly one storage unit. The total size of packages assigned to any storage unit must not exceed its capacity.

The capacity constraint is:

where represents the set of packages assigned to storage unit .

The main objective is:

subject to all package-assignment and capacity constraints.

The goal is therefore to store all packages using the minimum possible number of storage units while maintaining efficient space utilization.

3. Objectives:

The main objectives of the proposed warehouse storage optimization system are:

To model warehouse storage allocation as a Bin Packing Problem.

To minimize the number of storage units required.

To maximize storage-space utilization.

To distribute packages efficiently according to their sizes.

To design a feasible solution using greedy algorithms.

To compare First-Fit and Best-Fit approaches.

To analyze the time complexity of the algorithms.

To evaluate their performance for large-scale warehouses.

To identify the trade-off between optimality and computational efficiency.

To demonstrate the usefulness of Bin Packing algorithms in logistics applications.

4. Constraints:

Several constraints must be considered in warehouse storage optimization.

4.1 Storage Capacity

The total size of packages assigned to a storage unit cannot exceed its capacity.

where if package is assigned to storage unit , and otherwise.

4.2 Space Utilization

The system should minimize unused space in storage units. Better utilization allows more packages to be stored using fewer units.

Space utilization is calculated as:

4.3 Item Distribution

Packages of different sizes must be distributed efficiently. Large packages may occupy most of a storage unit's capacity, whereas smaller packages can often be combined to reduce unused space.

4.4 Package Compatibility

Real warehouses may have restrictions on which products can be stored together. For example, fragile products may require special storage, while incompatible materials may need to be separated.

4.5 Accessibility

Frequently requested packages should ideally be stored in easily accessible locations. This can reduce the time required for picking and improve warehouse productivity.

4.6 Weight Restrictions

Some storage units may have maximum weight limits. Therefore, the optimization system may need to consider both package size and package weight.

5. Mathematical Model

The Bin Packing Problem can be represented using binary decision variables.

Let:

Every package must be assigned to exactly one storage unit:

For each storage unit:

The objective is to minimize the number of storage units used.

This mathematical formulation ensures that every package is stored exactly once and that no storage unit exceeds its capacity.

6. Bin Packing Algorithms

The Bin Packing Problem is computationally difficult, particularly for large numbers of packages. Therefore, practical systems often use heuristic or approximation algorithms.

Two important greedy approaches are:

First-Fit

Best-Fit

A useful improvement is First-Fit Decreasing.

7. First-Fit Algorithm

The **First-Fit (FF)** algorithm processes packages one at a time. For each package, it examines the existing storage units in a fixed order and places the package in the **first storage unit where it fits**.

If the package does not fit into any existing storage unit, a new storage unit is created.

7.1 First-Fit Steps

Start with an empty list of storage units.

Select the next package.

Examine the existing storage units in order.

Place the package in the first unit with sufficient remaining capacity.

If no existing unit can accommodate the package, create a new unit.

Repeat until all packages have been allocated.

7.2 Example

Assume each storage unit has a capacity of **100 units**, and the package sizes are:

60, 40, 70, 30, 20, 50

First-Fit produces:

Storage Unit 1:

Storage Unit 2:

Storage Unit 3:

Therefore:

are required.

8. Best-Fit Algorithm

The **Best-Fit (BF)** algorithm attempts to place each package into the storage unit that will have the **smallest remaining capacity after the package is placed**, among all storage units where the package fits.

For example, if a package has size 30 and available capacities are 40, 60, and 80 units, Best-Fit chooses the storage unit with 40 units of available capacity because it leaves only 10 units unused.

8.1 Best-Fit Steps

Start with an empty set of storage units.

Select the next package.

Identify all storage units where the package fits.

Calculate the remaining capacity after placement.

Select the storage unit with the smallest remaining capacity.

If the package does not fit into any existing unit, create a new storage unit.

Continue until all packages are assigned.

Best-Fit attempts to reduce unused space in individual storage units, but it does not guarantee an optimal solution.

9. First-Fit Decreasing Algorithm

A useful improvement to First-Fit is **First-Fit Decreasing (FFD)**.

FFD operates in two stages:

Sort all packages in decreasing order of size.

Apply First-Fit to the sorted packages.

For example, consider:

20, 70, 40, 60, 30, 50

After sorting:

70, 60, 50, 40, 30, 20

First-Fit is then applied to this sequence.

Processing larger packages first generally produces better packing than processing packages in an arbitrary order.

Therefore, FFD provides a practical balance between solution quality and computational efficiency.

10. Pseudocode

10.1 First-Fit Decreasing

Algorithm First-Fit Decreasing

Input:

Package sizes $W[1..n]$

Storage unit capacity C

Output:

Storage units containing all packages

1. Sort W in decreasing order.
2. Create an empty list of storage units.
3. For each package w in W :

placed = false

For each storage unit b:

 If remaining_capacity(b) $\geq$ w:

 Place w in b

 placed = true

 Break

 If placed == false:

 Create a new storage unit

 Place w in the new storage unit

4. Return the storage units.

11. Worked Example

Consider a warehouse with storage-unit capacity:

Package sizes are:

After sorting in decreasing order:

Applying First-Fit Decreasing:

Storage Unit 1

Storage Unit 2

Storage Unit 3

Storage Unit 4

Therefore, the algorithm uses:

Total package size is:

Total available capacity is:

Therefore:

Unused capacity is:

Thus, the solution achieves **86.25% space utilization**.

12. Time Complexity Analysis

The computational complexity of the algorithms is important when they are applied to large warehouses.

12.1 First-Fit

In a straightforward implementation, each package may need to be compared with every existing storage unit.

If there are packages and storage units:

In the worst case, can be proportional to . Therefore:

is the worst-case time complexity for a simple implementation.

12.2 Best-Fit

A straightforward Best-Fit implementation may also examine all existing storage units for every package.

Therefore:

and, in the worst case:

12.3 First-Fit Decreasing

FFD first sorts the packages.

Sorting requires:

The subsequent First-Fit phase can require:

in a straightforward implementation.

Therefore, the overall worst-case complexity is:

because the quadratic packing phase dominates the sorting phase.

With appropriate data structures, the practical performance of these algorithms can be improved.

13. Optimality versus Computational Efficiency

The Bin Packing Problem is an **NP-hard optimization problem**. Finding the exact minimum number of storage units can become computationally expensive as the number of packages increases.

Therefore, there is an important trade-off between optimality and computational efficiency.

Method	Solution Quality	Computational Cost	Suitability
Exact optimization	Optimal	Very high for large inputs	Small problems
First-Fit	Good	Low to moderate	Large-scale applications
Best-Fit	Good	Low to moderate	Large-scale applications
First-Fit Decreasing	Generally better than basic First-Fit	Moderate	Large warehouses

Exact optimization can guarantee the optimal solution, but it may not be practical for very large datasets.

Greedy algorithms may produce a non-optimal solution, but they can generate a good solution much faster.

14. Performance in Large-Scale Warehouses

Large warehouses may handle thousands or millions of packages. In such environments, storage allocation must be performed efficiently.

Greedy algorithms are useful because they:

- Process packages quickly.
- Are simple to implement.
- Require relatively low computational resources.
- Can handle large numbers of packages.
- Can be adapted to changing warehouse conditions.
- Provide good storage utilization.

For example, a warehouse management system can collect package dimensions or weights, sort packages according to size, and apply FFD to generate an initial storage allocation.

This approach can be particularly useful in automated warehouses where packages continuously arrive and storage decisions must be made quickly.

15. Performance Evaluation Metrics

The performance of the storage optimization algorithm can be measured using several metrics.

15.1 Number of Storage Units

A smaller number generally indicates better packing efficiency.

15.2 Space Utilization

A higher percentage indicates better utilization of available capacity.

15.3 Wasted Capacity

A lower value indicates less unused storage space.

15.4 Execution Time

The time required to generate the storage allocation is important in real-time warehouse environments.

15.5 Scalability

The algorithm should maintain acceptable performance as the number of packages increases from hundreds to thousands or millions.

16. Advantages of the Proposed Solution

The First-Fit Decreasing approach provides several advantages.

16.1 Good Space Utilization

Sorting packages by decreasing size generally allows large packages to be allocated first and smaller packages to fill the remaining spaces.

16.2 Fast Execution

Greedy methods are significantly faster than many exact optimization approaches for large datasets.

16.3 Simple Implementation

The algorithms are easy to understand, implement, test, and maintain.

16.4 Scalability

The approach can be applied to large numbers of packages.

16.5 Flexibility

The method can be adapted to different package sizes and storage capacities.

16.6 Reduced Storage Requirements

Better packing can reduce the number of storage units required, potentially reducing warehouse space requirements.

16.7 Practical Application

The algorithm can be integrated into warehouse management systems and automated storage systems.

17. Limitations

Although greedy algorithms are efficient, they do not always produce the optimal solution.

The result may depend on the order in which packages are processed. Different package sequences can produce different numbers of storage units.

Furthermore, the basic one-dimensional Bin Packing model does not completely represent the physical conditions of a real warehouse.

Practical systems may need to consider:

Length

Width

Height

Volume

Package weight

Fragility

Product compatibility

Safety requirements

Storage location

Picking frequency

Travel distance

Temperature requirements

Therefore, a real warehouse may require a **multi-dimensional or constrained Bin Packing model** rather than a simple one-dimensional model.

18. Possible Improvements

The basic greedy approach can be improved in several ways.

18.1 Multi-Dimensional Packing

Instead of considering only one package measurement, the algorithm can consider:

Length

Width

Height

Volume

Weight

This provides a more realistic representation of physical storage.

18.2 Priority-Based Storage

Frequently requested products can be assigned to easily accessible locations to reduce picking time.

18.3 Compatibility Constraints

The algorithm can prevent incompatible products from being assigned to the same storage area.

18.4 Dynamic Reallocation

When packages are removed from storage, the system can periodically reorganize the remaining packages to reduce unused capacity.

18.5 Hybrid Optimization

A greedy algorithm can generate an initial solution, after which local search or another optimization technique can improve the allocation.

This approach provides a balance between computational efficiency and solution quality.

19. Application in Logistics

Bin Packing algorithms have several practical applications in logistics and warehouse management, including:

Storage-unit allocation

Warehouse space planning

Container loading

Pallet loading

Vehicle loading

Distribution-center organization

Automated storage systems

For example, when a warehouse receives a large batch of packages, FFD can quickly generate an initial allocation. The warehouse management system can then assign physical storage locations based on the calculated groups.

20. Trade-Off Analysis

The choice of algorithm depends on the requirements of the warehouse.

If the warehouse contains a small number of packages and obtaining the absolute minimum number of storage units is critical, an exact optimization method may be appropriate.

If the warehouse processes a very large number of packages, computational efficiency becomes more important. In such cases, First-Fit or Best-Fit can provide fast solutions.

First-Fit Decreasing provides a strong practical compromise because it improves the package-processing order through sorting while retaining the simple implementation of First-Fit.

The main trade-off can therefore be represented as:

A practical warehouse system should select an algorithm based on package volume, response-time requirements, storage constraints, and the required level of optimization.

21. Expected Results

The proposed approach is expected to provide:

Efficient allocation of packages.

Reduced unused storage capacity.

Fewer storage units compared with an arbitrary allocation.

Faster computation than exact optimization for large datasets.

Improved scalability for large warehouses.

A practical method for automated storage planning.

The exact performance will depend on the number and distribution of package sizes, storage-unit capacity, and the implementation of the algorithm.

22. Conclusion

Smart warehouse storage optimization can be effectively modeled as a **Bin Packing Problem**, where packages represent items and storage units represent fixed-capacity bins. The main objective is to minimize the number of storage units while ensuring that no storage unit exceeds its capacity.

Greedy algorithms such as **First-Fit** and **Best-Fit** provide fast and practical solutions. Among these methods, **First-Fit Decreasing (FFD)** is particularly suitable for large-scale warehouse applications because sorting packages in decreasing order generally improves packing efficiency while maintaining reasonable computational performance.

A straightforward implementation of First-Fit, Best-Fit, and FFD can have a worst-case time complexity of $O(n \log n)$. Although these algorithms do not guarantee an optimal solution, they provide a good balance between solution quality and computational efficiency.

For real-world warehouse applications, the basic Bin Packing model can be extended by considering package dimensions, weight, compatibility, accessibility, picking frequency, and safety requirements. Combining greedy algorithms with additional optimization techniques can further improve the quality of storage allocation.

Overall, Bin Packing algorithms provide an effective foundation for smart warehouse storage optimization. Their simplicity, speed, scalability, and ability to improve space utilization make them highly suitable for modern logistics and warehouse management systems.

23. References

Huang, X., Wang, X., Pu, Y., et al. (2026). *Optimizing One-Dimensional Bin Packing for Conveyor Belt Logistics via Pointer-Network-Based A2C and Heuristics*. **Scientific Reports**.

Galambos, G., Martello, S., & Vigo, D. (2025). *An Updated Overview of Bin Packing Approximation Algorithms*. **Springer**.

Rakotonirainy, R. G., & Netshitungulu, F. (2025). *Optimising Bin Packing in Transportation Logistics: Heuristic Approaches and Instance Space Analysis*. **South African Journal of Industrial Engineering**.

Mantzou, T., & Dimitriadis, S. (2025). *Generalized Bin Packing and Related Problems: A Systematic Literature Review*. **Journal of Industrial Engineering and Management**.

Tsang, Y. P., Mo, D. Y., Chung, K. T., & Lee, C. K. M. (2025). *A Deep Reinforcement Learning Approach for Online and Concurrent 3D Bin Packing Optimisation*. **Computers in Industry**.

IMPLEMENTATION CODE:

```
# SMART WAREHOUSE STORAGE OPTIMIZATION
```

```
# Bin Packing using Greedy Algorithms
```

```
def first_fit(packages, capacity):
```

```
    bins = []
```

```
    for package in packages:
```

```
        placed = False
```

```
        for i in range(len(bins)):
```

```
            if sum(bins[i]) + package <= capacity:
```

```
                bins[i].append(package)
```

```
                placed = True
```

```
                break
```

```
        if placed == False:
```

```
            bins.append([package])
```

```
    return bins
```

```
def best_fit(packages, capacity):
```

```
    bins = []
```

```
    for package in packages:
```

```
        best_bin = -1
```

```

min_space = capacity + 1
for i in range(len(bins)):
    used = sum(bins[i])
    remaining = capacity - used
    if package <= remaining:
        space_left = remaining - package
        if space_left < min_space:
            min_space = space_left
            best_bin = i
    if best_bin != -1:
        bins[best_bin].append(package)
    else:
        bins.append([package])
return bins

def first_fit_decreasing(packages, capacity):
    sorted_packages = sorted(packages, reverse=True)
    return first_fit(sorted_packages, capacity)

def display_result(name, bins, capacity):
    print()
    print(name)
    print("-----")
    total_used = 0

    for i in range(len(bins)):
        used = sum(bins[i])
        remaining = capacity - used
        total_used = total_used + used
        print("Storage Unit", i + 1, ":", bins[i])
        print("Used Capacity    :", used)
        print("Remaining Capacity :", remaining)
        print()
    total_capacity = len(bins) * capacity

```



```

utilization = (total_used / total_capacity) * 100

print("Storage Units Used :", len(bins))

print("Space Utilization : {:.2f}%".format(utilization))

print("Wasted Capacity   :", total_capacity - total_used)


# MAIN PROGRAM

capacity = 100

packages = [45, 25, 70, 30, 55, 20, 40, 60]

print("SMART WAREHOUSE STORAGE OPTIMIZATION")

print("=====")

print("Storage Unit Capacity:", capacity)

print("Package Sizes:", packages)

# First-Fit

ff_result = first_fit(packages, capacity)

# Best-Fit

bf_result = best_fit(packages, capacity)

# First-Fit Decreasing

ffd_result = first_fit_decreasing(packages, capacity)

# Display results

display_result("FIRST-FIT", ff_result, capacity)

display_result("BEST-FIT", bf_result, capacity)

display_result("FIRST-FIT DECREASING", ffd_result, capacity)

```

OUTPUT:

```
main.py
1 # SMART WAREHOUSE STORAGE OPTIMIZATION
2 # Bin Packing using Greedy Algorithms
3
4
5 def first_fit(packages, capacity):
6     bins = []
7
8     for package in packages:
9         placed = False
10
11         for i in range(len(bins)):
12             if sum(bins[i]) + package <= capacity:
13                 bins[i].append(package)
14                 placed = True
15                 break
16
17         if placed == False:
18             bins.append([package])
19
20     return bins
21
22
23 def best_fit(packages, capacity):
24     bins = []
25
26     for package in packages:
27         best_bin = -1
28         min_space = capacity + 1
```

input

Storage Units Used : 4
Space Utilization : 86.25%
Wasted Capacity : 55

...Program finished with exit code 0
Press ENTER to exit console.